Table des matières

1 Introduction **1**

2 Questions théoriques **1**
2.1 Exercice 3 1
2.2 Exercice 4 1

3 Analyse théorique **3**

1 Introduction

Ce rapport présente un compte rendu du projet réalisé dans le cadre de l'UE COMPLEX, dont le but était d'appliquer l'algo du branch and bound vu en cours pour résoudre le problème de couverture de graphe et d'évaluer les performances de différentes variantes de l'algo. Nous avons choisi C comme langage pour réaliser des tests de performances fiables et d'exploiter un contrôle fin sur les structures de données, permettant de traiter efficacement des graphes de tailles élevées et de générer des mesures expérimentales représentatives. Comparé à des langages de haut niveau comme Python, C minimise l'overhead et permet d'avoir les performances brutes des algos étudiés. Nous commencerons par répondre aux questions théoriques dans un premier temps puis on finira par comparer les différents algos avec l'aide de différents graphiques illustrant leur performance.

2 Questions théoriques

2.1 Exercice 3

1.

Posont le graphe G = (V, E) avec V = $V_0 \cup V_1 \cup V_2$ avec :
- $V_0 = \{s_0\}$
- $V_1 = \{s_1, s_2, s_3, s_4, s_5, s_6\}$
- $V_2 = \{s_7, s_8, s_9, s_{10}, s_{11}, s_{12}, s_{13}, s_{14}\}$

Et E = $E_0 \cup E_1 \cup E_2$ avec :
- $E_0 = \{\{s_0, s_i\}\ \forall i \in \{1, \ldots, 6\}\}$
- $E_1 = \{\{s_i, s_j\}\ \forall i \in \{1, \ldots, 6\}, \forall j \in \{7, \ldots, 14\}\}$
- $E_2 = \{\{s_i, s_{i+1}\}\ \forall i \in \{7, \ldots, 13\}\}$

On peut remarquer que dans ce graphe :

$$\begin{cases} deg(s_0) = 6 & \\ deg(s) = 9 & \text{si } s \in V_1 \\ deg(s) < 9 & \text{si } s \in V_2 \end{cases}$$

L'algorithme glouton va donc prendre tout les sommets de V_1 puisque ils sont tous de degré maximal, puis les sommets $s_8, s_{10}, s_{12}, s_{13}$. Ceci nous donne donc une couverture de taille 10. Or une autre solution de taille inférieure serait de ne que prendre les sommets de V_2, puis s_0, ce qui donne une couverture de taille 9. Donc l'algorithme glouton n'est pas optimal, et n'est pas 1-approché.

2.2 Exercice 4

Partie 2

1.b_1 : Soit S défini comme

$$S = \{(v, e) \mid (v, e) \in C \times E\}.$$

On a $|S| \geq m$ par définition de C.

De plus,

$$|S| = \sum_{v \in C} \deg(v),$$

or par définition du degré maximal Δ, on a $\deg(v) \leq \Delta$ pour tout $v \in C$.

Ainsi,

$$|S| \leq \sum_{v \in C} \Delta = |C| \cdot \Delta.$$

En combinant les deux inégalités, on obtient :

$$|C| \cdot \Delta \geq m \quad \Rightarrow \quad |C| \geq \frac{m}{\Delta}.$$

Comme $|C| \in \mathbb{N}$, on a finalement :

$$|C| \geq \left\lceil \frac{m}{\Delta} \right\rceil.$$

b_2 : Soit un graphe $G = (V, E)$ et une couverture d'arêtes $C \subseteq V$. Par définition, C est une couverture si et seulement si

$$\forall (u, v) \in E, \quad u \in C \text{ ou } v \in C.$$

Ainsi, pour toute arête $(u, v) \in M \subseteq E$, on a nécessairement que

$$u \in C \quad \text{ou} \quad v \in C,$$

car chaque arête doit être couverte par au moins un sommet de C.
On en déduit donc que

$$|C| \geq |M|.$$

b_3 : On rappelle qu'un graphe complet G à n sommets possède $\dfrac{n(n-1)}{2}$ arêtes.

En prenant le sous-graphe G' ayant pour sommets C, celui-ci possède au plus

$$\frac{k(k-1)}{2} \quad \text{avec } k = |C|$$

arêtes entre les sommets de C.

Entre les sommets de $V \setminus C$ et ceux de C, on a au plus $k(n-k)$ arêtes.

Par définition d'un couplage, on ne peut pas avoir d'arêtes entre sommets de $V \setminus C$. Ainsi :

$$m \leq \frac{k(k-1)}{2} + k(n-k).$$

En fixant m et en multipliant par -2, puis en développant, cette inégalité revient à résoudre l'inéquation du second degré :

$$k^2 - (2n-1)k + 2m \leq 0.$$

Le discriminant vaut :

$$\Delta = (2n-1)^2 - 8m.$$

Celui-ci est bien positif, car :

$$m \leq \frac{n(n-1)}{2} \quad \Longleftrightarrow \quad 8m \leq 4(n^2 - n),$$

et puisque $(2n-1)^2 = 4n^2 - 2n$, cette inégalité est vérifiée pour tout $n \geq 0$.

La solution de l'inéquation est donc :

$$k \in \left[\frac{(2n-1) - \sqrt{(2n-1)^2 - 8m}}{2} \, , \, \frac{(2n-1) + \sqrt{(2n-1)^2 - 8m}}{2} \right].$$

On en conclut que :

$$|C| \geq \frac{(2n-1) - \sqrt{(2n-1)^2 - 8m}}{2}.$$

Partie 3

3. Soit $G = (V, E)$ un graphe, u un sommet de degré 1 et C une couverture optimale de G.
Comme u est de degré 1, il existe un sommet $v \in V$ tel que $(u, v) \in E$.

Si $u \notin C$, il n'y a rien à montrer.

Supposons que $u \in C$. Comme C est une couverture *optimale*, on doit avoir $v \notin C$ (sinon, on pourrait retirer u sans perdre la propriété de couverture).
Or, comme $\deg(u) = 1$, le sommet u ne couvre que l'arête (u, v).
En considérant :

$$C' = (C \setminus \{u\}) \cup \{v\},$$

on a :

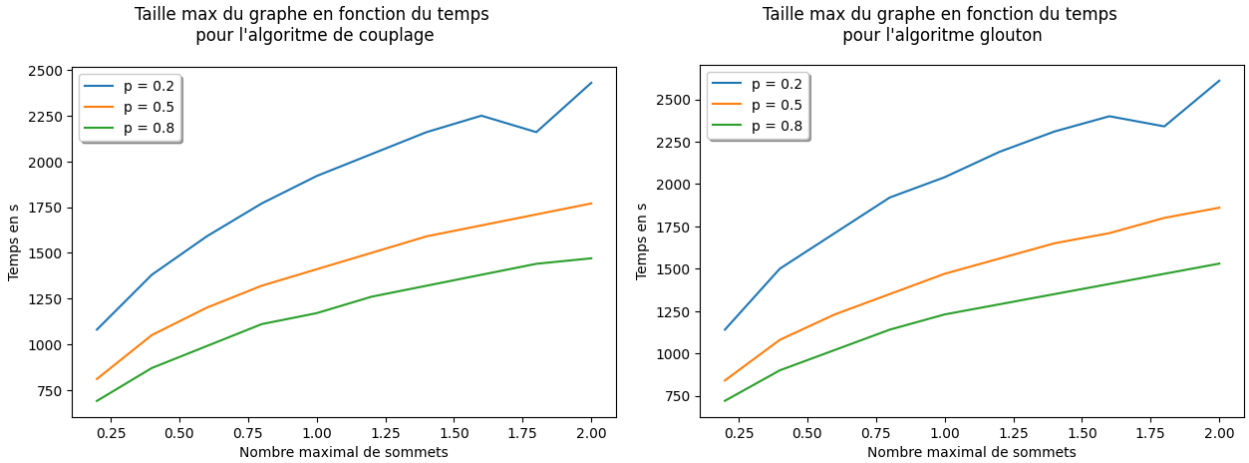
$$|C'| = |C| \quad \text{et} \quad C' \text{ couvre bien toutes les arêtes de } G.$$

Ainsi, C' est également une couverture optimale de G.
Il est donc toujours possible de trouver une couverture optimale qui ne contient pas le sommet u de degré 1.

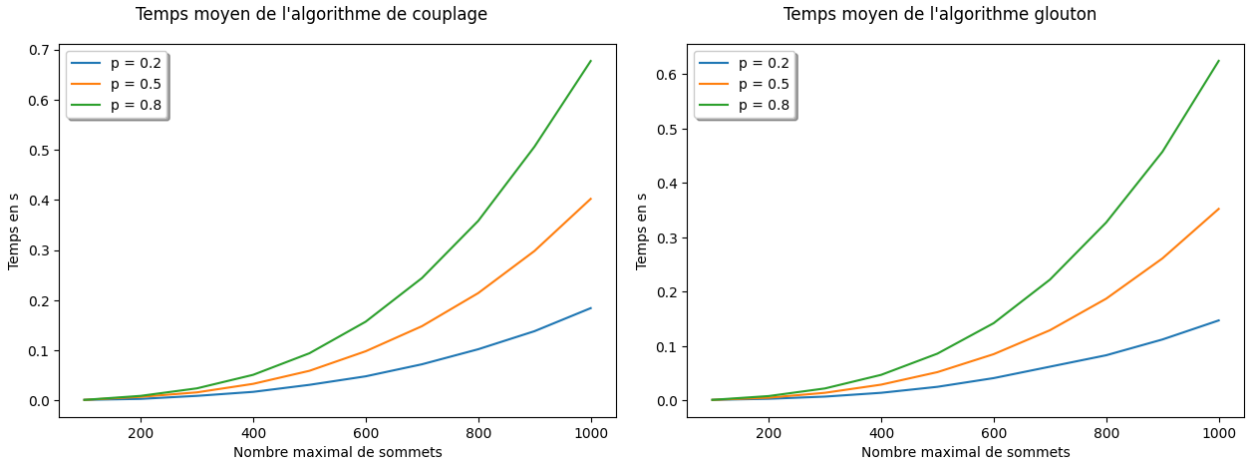
3 Analyse théorique

Exercice 3

2.



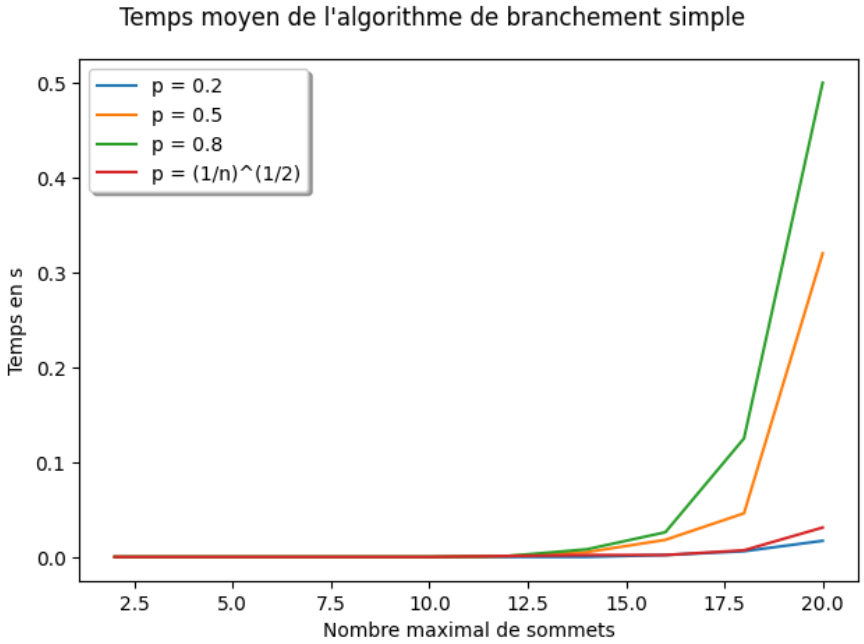
On peut observer que dans les deux algorithmes, la taille maximal est inversement proportionnelle à la quantité d'arêtes présentes dans le graphe. Ceci est dû au fait que plus le graphe est dense, plus on a d'arêtes à retirer pour chaque suppression de sommet.



Le temps d'éxécution est exponentiel en fonction du nombre de sommets, ceci est dû au fait que la complexité de la fonction de suppression d'un sommet du graphe ne dépend pas que du nombre de sommets, mais aussi du nombre d'arêtes, qui est quadratique en fonction du nombre de sommets du graphe.

Exercice 4

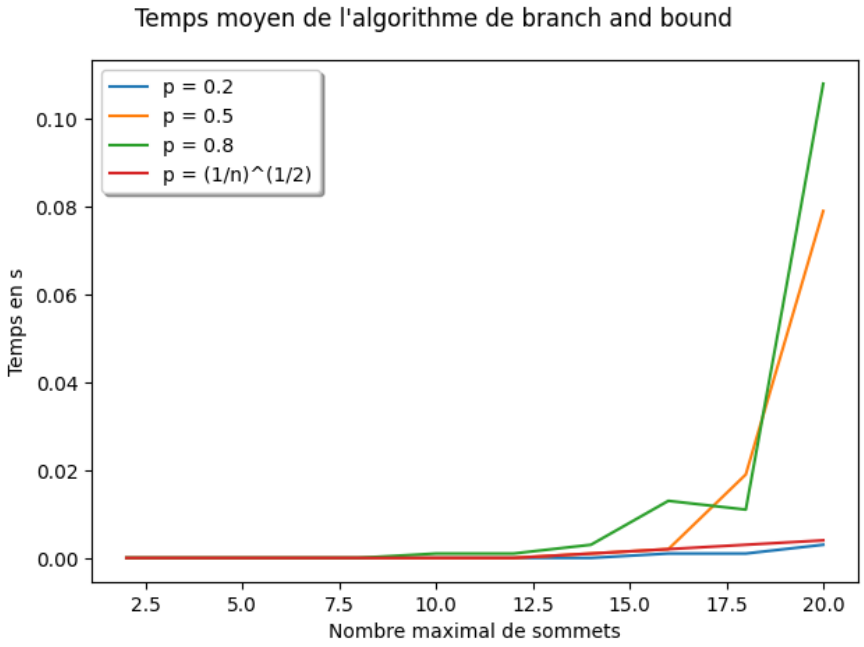
Partie 1



On voit que la vitesse de l'algorithme est corrélé positivement avec la densité du graphe, plus le graphe est dense, donc plus p est grand, plus l'algorithme est lent. Ceci est directement lié au fait que le nombre de noeuds visité dans un algorithme de branchement est égal à 2^m, avec m le nombre d'arêtes du graphe. On a donc un temps d'exécution exponentiel en fonction du nombre de sommets.

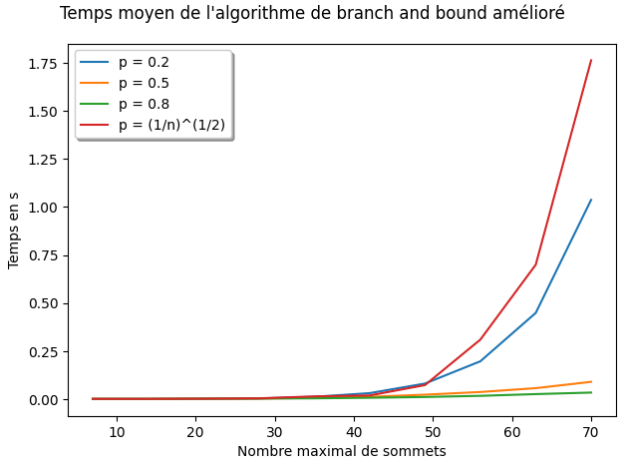
Partie 2

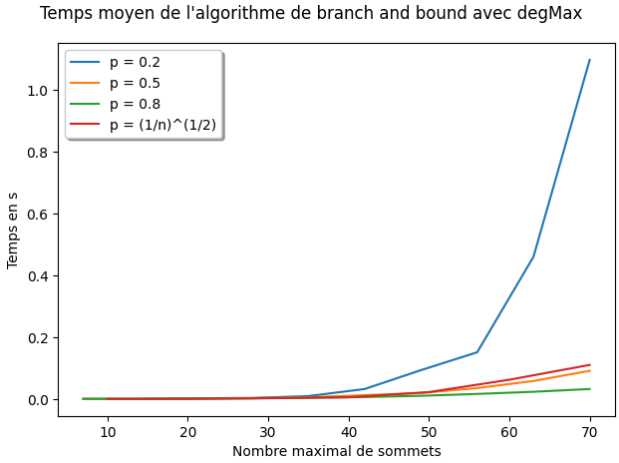
2.

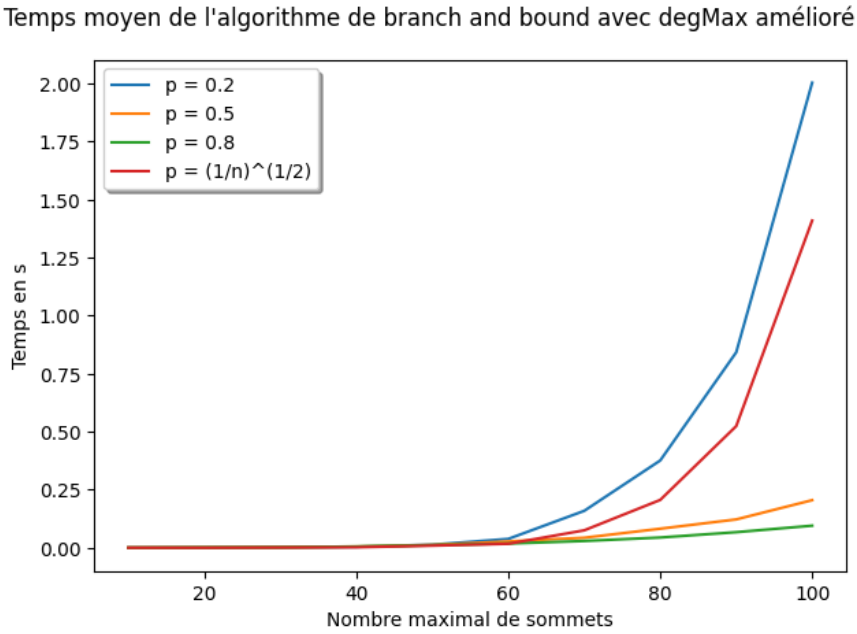


Similairement à la version sans borne, on a une corrélation positive entre p et la vitesse de l'algorithme. On voit aussi que malgé l'ajout des bornes, les performances de l'algorithme de branch and bound sont similaires à celles sans bornes. Nos bornes ne sont donc pas assez restrictives pour permettre un bon élagage de l'arbre.

Partie 3







Cependant, lorsque on retire u ainsi que tout les sommets voisins de u quand on examine le graphe avec le sommet v dans la couverture, on obtient de bien meilleures performances, avec des temps d'exécutions qui sont en-dessous de 0,1 secondes tant que le nombre de sommet est inférieur à 50, ce qui est bien meilleur quand comparé aux deux versions précédentes.
Cette fois-ci, on a une corrélation négative entre p et la vitesse de l'algorithme. Ceci est dû au fait que si le graphe n'est pas dense, retirer un sommet ainsi que tout ses voisins ne va pas retirer beaucoup de sommets, et va donc forcer l'algorithme a explorer plus de noeuds.
On peut aussi voir que notre heuristique de toujours sélectionner u de sorte à ce qu'il soit de degré maximal est une bonne heuristique, les temps de calculs pour des graphes avec 70 sommets étant d'environ 2 secondes dans le pire cas pour le branch and bound amélioré, alors qu'ils sont d'environ 1 seconde en prenant le sommet de degré maximal, qui est réduit à 0,25 quand on y ajoute le retrait des sommets de degré 1.